

Pseudocode:

BEGIN

Initialize weight and bias randomly

FOR epoch in range (max-epochs)

For each input sample

* Forward pass

compute $z = w^T x + b$

apply activation to get A

compute output prediction

* compute loss

loss = cost(y, tree, y-pred)

* backward pass

compute gradients dw, db using chain

rule update parameters:

$$w = w - \eta \cdot dw$$

$$b = b - \eta \cdot db$$

END for

Print after epoch

ENDFOR

END

09-09-25
ex: 6

6. Implement gradient descent & backpropagation in deep neural networks

Aim: To implement gradient descent and backpropagation in deep neural networks.

Objectives:

- * To understand gradient descent in an optimized method.
- * To implement backpropagation in deep neural networks to update weights.
- * To implement a simple neural network for classification task.
- * To observe how loss decrease with iteration.

OBSERVATION

- * Loss decreases as number of iterations increases.
- * Weights and bias object to minimize error.
- * Backpropagation ensures errors are effectively distributed layer by layer.
- * Learning rate (α) greatly influences convergence speed.

Result:

Therefore implementation of gradient descent and backpropagation in neural network.

output:

epoch 0 - loss : 0.7198

epoch 500 - loss : 0.6869

epoch 1000 - loss : 0.6755

epoch 1500 - loss : 0.6402

epoch 2000 - loss : 0.5343

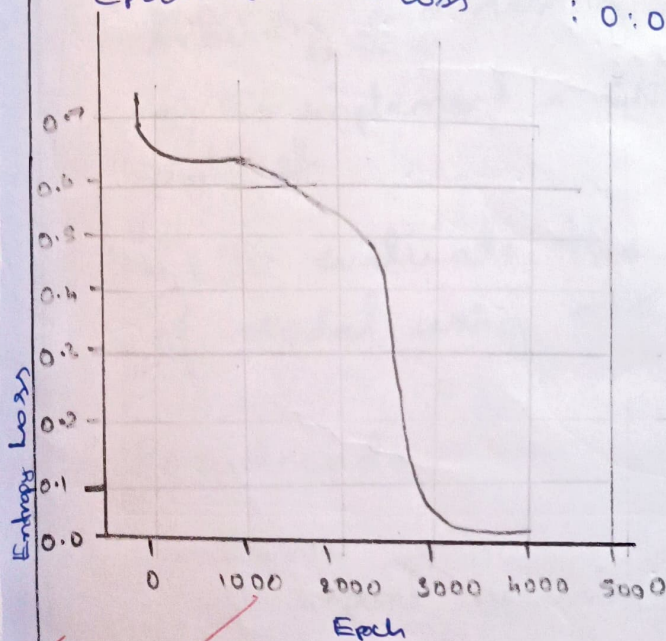
epoch 2500 - loss : 0.2707

epoch 3000 - loss : 0.0883

epoch 3500 - loss : 0.0417

epoch 4000 - loss : 0.0153

epoch 4500 - loss : 0.0177



Prediction

tensor([0.],
[1.],
[1.],
[0.]])

Ground Truth

tensor([[0.],
[1.],
[1.],
[0.]])